

Municipal Financial Management System (MFMS)

Technical Report

Course: PAP521S – Programming in Practice

Repository: SaaraHango/Hango-Financial-Management-System

1. Introduction

Municipalities manage large amounts of financial information, including staff salaries, departmental budgets, supplier details and physical assets. When this is handled manually or in disconnected files, it is slow, error-prone and hard to report on. This report describes the foundation version of a Municipal Financial Management System (MFMS), developed in ANSI C (C99) with Visual Studio Code, GCC, and GitHub. The system is the first stage of a larger application.

2. Problem Description

The municipality needs a simple program that can:

- record and look up employees and calculate their salary information;
- track departmental budgets and spending, and flag departments that overspend;
- keep a register of suppliers and their contact details;
- maintain a register of municipal assets such as vehicles, computers and buildings;
- produce summary reports from the stored data.

3. System Objectives

1. Provide a clear, menu-driven console interface.
2. Store records in arrays using suitable data types.
3. Validate user input to prevent obviously invalid data.
4. Support adding, displaying and searching records in each module.
5. Perform salary and budget calculations using arithmetic, relational and logical operators.
6. Generate employee, budget, supplier and asset reports.
7. Organise the program into logical functions and modules rather than one large main().
8. Demonstrate collaborative development through Git and GitHub.

4. System Features

Main menu. Six options (Employee, Budget, Supplier, Asset, Reports, Exit) in a loop that repeats until the user exits. Invalid choices produce a message, and the menu is shown again.

Employee management. Add and display employees and search by [ID / name]. Each record holds employee ID, name, department, basic salary, housing allowance, transport allowance.

Budget management. Capture a department's allocated budget and expenditure, calculate the remaining budget, and set a status of WITHIN BUDGET or EXCEEDED. Departments over budget can be listed.

Supplier management. Store supplier ID, name, email, telephone and town/location. Suppliers can be added, displayed and searched.

Asset management. Store asset ID, name, type, purchase value, department and condition. Assets can be displayed and searched.

Reports.

- *Employee report*: total employees, average, highest and lowest salary.
- *Budget report*: total allocated budget, total expenditure, remaining budget and departments exceeding budget.
- *Supplier report*: all registered suppliers.
- *Asset report*: all registered assets.

Data validation. The system rejects negative salaries and budgets, empty names and non-numeric input where numbers are expected, and it handles invalid menu choices.

5. Program Design

Modular structure. The code is split into modules, each with a .c source file and a .h header:

File	Responsibility
main.c	Main menu and program flow
employees.c/.h	Employee records, search, salary calculation
budget.c/.h	Budgets, expenditure, balance and status
suppliers.c/.h	Supplier records and search
assets.c/.h	Asset register and search
reports.c/.h	Report generation
README.md	Documentation and instructions

Data storage. Records are stored in arrays, with a counter per module tracking the number of records in use. Fixed maximum sizes are defined with define constants.

Key functions. displayMenu(), addEmployee(), displayEmployees(), searchEmployee(), calculateSalary(), addSupplier(), displaySuppliers(), calculateBudget(), displayReports(), plus validation helpers. Parameters are passed to functions and results returned so that calculation logic is separated from input and output.

String handling. strlen() is used to detect empty input and check lengths, strcmp() to compare IDs and names in searches, strcpy() to store text into records, and strcat().

Control structures. switch handles menu selection, if/else handles budget status and validation, and for/while loops handle menu repetition, record traversal and searching.

Processing flow. The program follows the cycle INPUT → PROCESSING → STORAGE → SEARCH → CALCULATION → OUTPUT.

6. Challenges Encountered

- Coordinating seven developers. Merge conflicts and inconsistent coding styles appeared when modules were integrated.
- Input handling in C. Leftover characters in the input buffer broke later scanf() calls, and non-numeric input could cause infinite loops.
- Strings with spaces. Names such as "Finance Department" were truncated by scanf("%s").
- Sharing data between modules without global-variable clutter.
- Searching and comparing strings correctly, including case differences.
- Fixed-size arrays limiting how many records can be stored.

7. Solutions Implemented

- Used Git branches per member and regular commits, with a designated member coordinating merge, so each person has visible contributions.
- Agreed on shared naming conventions and header files up front.
- Wrote reusable validation functions that read a line with `fgets()`, then convert and check it, instead of reading raw `scanf()` input.
- Used `fgets()` with newline removal for names and departments.
- Passed arrays and counters as function parameters to keep modules independent.
- Defined array size limits as constants and reported a clear "storage full" message when reached.
- Tested each module on its own, then the integrated system, with valid, invalid and boundary inputs.

8. Conclusion

The MFMS foundation version covers the required modules: employees, budgets, suppliers, assets and reports. It uses the core C concepts from the first half of the course, with validation and a modular layout. The main limitation is that data is held in memory and lost when the program closes. This, and the fixed array sizes, can be addressed in Project B, for example with file handling, dynamic memory and further refactoring, building directly on this codebase.